Predictive Learning — Homework 2

VC-theory and its Applications

Name: 蔣碩文

ID: 61547101S

Problem 1 (model selection) and Problem 2 (trading international mutual funds)

Code and data: https://github.com/tc9305/predict_learning/tree/main/HW2

Problem 1: Model selection for trigonometric polynomial regression

Experimental setup

I follow the textbook's protocol for comparing model selection methods: generate many random training sets, let each method choose a model complexity, and record the selected degrees of freedom (DoF) and the prediction risk of the chosen model on a large independent test set. The approximating functions are the trigonometric polynomials defined in the course lecture notes,

$$f_m(x) = w_0 + \sum_{i=1}^{m} \big[v_i \sin(ix) + w_i \cos(ix)\big], \qquad \text{DoF} = 2m + 1,$$

fitted by least squares. For a model that is linear in its parameters, the VC-dimension equals the number of free parameters, so here $h = \text{DoF} = 2m + 1$. I draw inputs from $x \sim U[0, 1]$ and use the two data sets of the textbook experiment:

- **Pure noise:** $y \sim N(0, 1)$. The true function is $f(x) = 0$, so the best model is a constant (DoF $= 1$).
- **Sine-squared:** $y = \sin^2(2\pi x) + \varepsilon$, $\varepsilon \sim N(0, 0.2^2)$. The target equals $\frac{1}{2} - \frac{1}{2}\cos(4\pi x)$, and its frequency $4\pi \approx 12.6$ matches none of the integer frequencies i in the basis. Every model in the family approximates the target, so this data set has no single correct DoF. The textbook's version of this experiment, which uses algebraic polynomials, has the same property.

I run 300 independent realizations for $n = 30$ (candidate DoF $1, 3, \ldots, 19$) and $n = 100$ (candidate DoF $1, 3, \ldots, 25$). In each realization, every method selects a DoF. I refit the model with that DoF and measure its *prediction risk* as the mean squared error between the fitted model and the *noise-free* target on 2000 independent test points. No model can predict the noise, so this risk measures the distance between the selected model and the truth.

On $[0, 1]$ the functions $\sin(ix)$ and $\cos(ix)$ with small i have similar shapes, and the design matrix becomes ill-conditioned as m grows. In 200 random samples with $n = 30$, the matrices with DoF 17 and 19 had numerical rank below their column count every time, and the matrix with DoF 15 in 8 of the 200 (Appendix, Check 1). In those cases the least-squares routine returns the minimum-norm solution. I keep the nominal DoF $= 2m + 1$ for the penalization factors.

Model selection methods

Training error falls as complexity grows, so it cannot guide model selection by itself. In one sine-squared sample with $n = 30$, the training MSE fell from 0.107 at DoF $= 1$ to 0.019 at DoF $= 7$ and 0.0092 at DoF $= 19$, below the noise variance $\sigma^2 = 0.04$. I compare the four methods of the

textbook experiment. Three of them multiply the training error R_{emp} by a penalization factor $r(p)$, where $p = \text{DoF}/n$, and select the DoF that minimizes $r(p)\,R_{\text{emp}}$:

Method	Penalization factor $r(p)$
Akaike's final prediction error (fpe)	$(1+p)/(1-p)$
Generalized cross-validation (gcv)	$(1-p)^{-2}$
VC bound for regression (vc)	$\left(1-\sqrt{p-p\ln p+\ln n/(2n)}\right)_+^{-1}$

For vc, I set the factor to infinity when the term under the square root reaches 1, which excludes that model. With $n = 30$ and $p = 0.1$, fpe gives 1.22, gcv 1.23, and vc 2.65, so vc charges about twice as much as the other two. At $p = 0.5$ the gap widens: fpe gives 3.0, gcv 4.0, and vc 20.2.

The fourth method, leave-one-out cross-validation (cv), estimates prediction risk from held-out data. I compute the leave-one-out error from a single fit with the leverage formula $e_i/(1-h_{ii})$, where e_i is the ordinary residual and h_{ii} the i-th diagonal entry of the hat matrix. The formula assumes that deleting one sample leaves the fitted subspace unchanged. With this ill-conditioned basis, deleting a sample can change the numerical rank, and rounding errors grow. For one sample with DoF = 19 and rank 17, the formula gave 74.2 while 30 explicit refits gave 0.54; at DoF = 15, with full rank, the two differed in the third decimal place (0.4951 vs. 0.4944). To avoid these errors, I refit n times whenever the matrix is rank deficient. As a check, I repeated the cv selection with explicit refits for every candidate DoF: in each of the four settings, cv chose the same DoF in all 300 realizations (Appendix, Check 3).

Results

Figures 1 and 2 present the results in the textbook's box-plot format. Each box spans the 25th to 75th percentiles with a line at the median, and the whiskers reach the 5th and 95th percentiles. The risk panels use a logarithmic scale, and a dashed line in the pure-noise DoF panels marks the best DoF.

	Pure noise, $n = 30$			Pure noise, $n = 100$		
Method	DoF = 1	risk p50	risk p95	DoF = 1	risk p50	risk p95
fpe	61.3%	0.0600	169.1	78.3%	0.0075	0.155
gcv	73.0%	0.0335	2.82	81.0%	0.0074	0.130
vc	99.0%	0.0165	0.144	100.0%	0.0049	0.0381
cv	76.0%	0.0317	0.492	79.3%	0.0077	0.152

	Sine-squared, $n = 30$			Sine-squared, $n = 100$		
Method	DoF p50 (p95)	risk p50	risk p95	DoF p50 (p95)	risk p50	risk p95
fpe	7 (15)	0.0303	413.3	9 (15)	0.0052	0.0192
gcv	7 (13)	0.0249	3.41	9 (13)	0.0050	0.0179
vc	7 (9)	0.0210	0.232	7 (9)	0.0051	0.0108
cv	7 (9)	0.0265	0.390	9 (15)	0.0050	0.0195

DoF = 1: share of the 300 realizations that selected the constant model; *p50*, *p95*: median and 95th percentile over the 300 realizations.

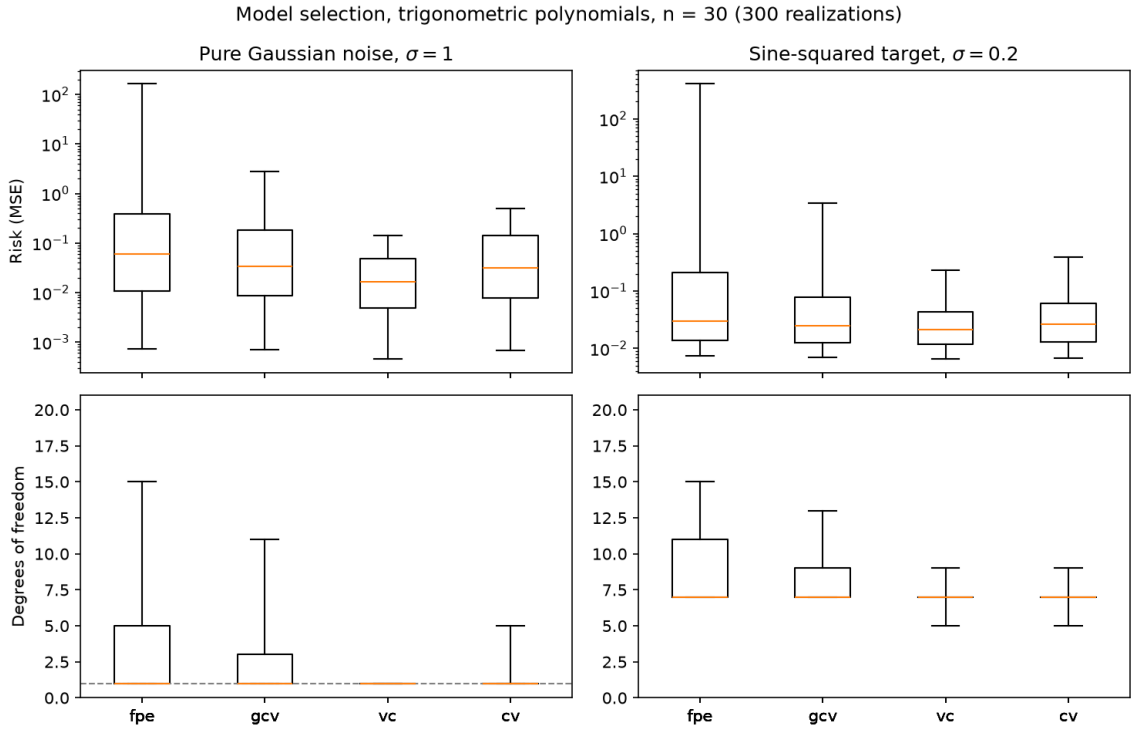


Figure 1: Model selection results for $n = 30$ (300 realizations). Top: prediction risk (log scale); bottom: selected DoF. Left: pure noise; right: sine-squared.

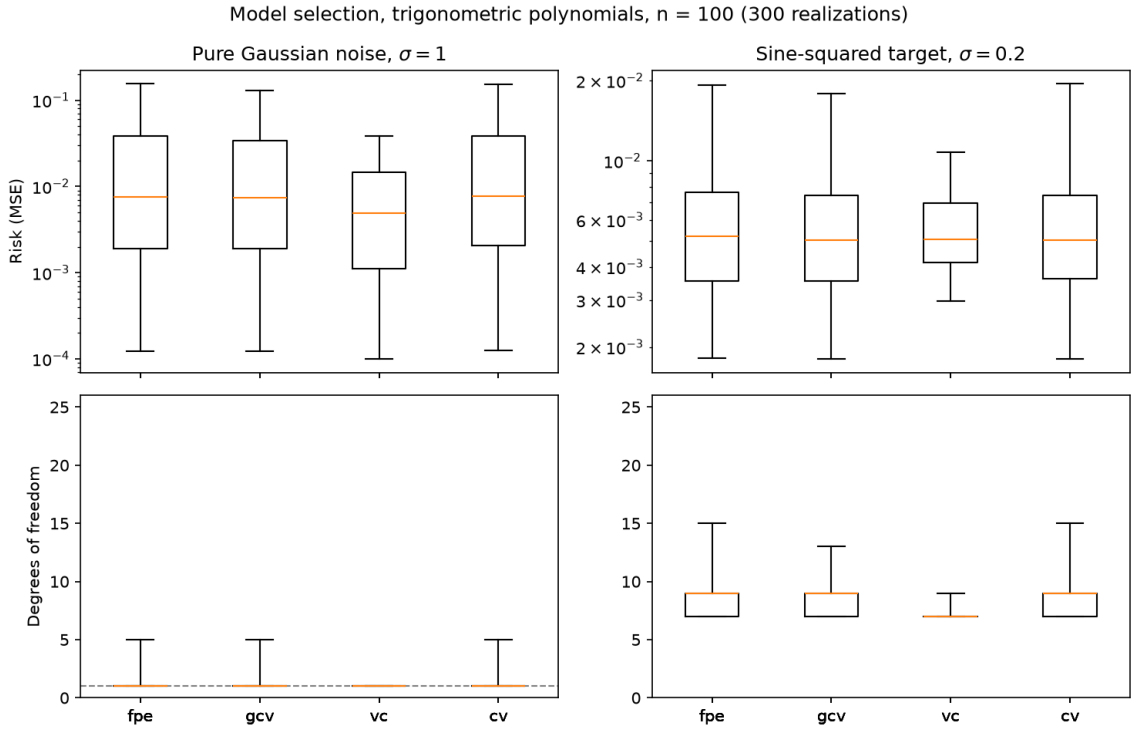


Figure 2: Model selection results for $n = 100$ (300 realizations), same layout as Figure 1.

Small samples ($n = 30$). For pure noise, the share of realizations that selected the constant model follows the strength of the penalty: vc (99.0%) > cv (76.0%) > gcv (73.0%) > fpe (61.3%). The 95th-percentile risk of fpe (169.1) is about 1200 times that of vc (0.144), while their medians differ by a factor of 3.6. In its worst 5% of realizations, fpe picked models with DoF 11–19, which swing far from zero between the training points; 12 of the 15 worst vc selections had DoF 1. For sine-squared, all four methods have a median DoF of 7 and the methods separate in the upper tail: the 95th-percentile DoF is 15 for fpe, 13 for gcv, and 9 for vc and cv. The 95th-percentile risk of fpe reaches 413, against 0.23 for vc.

These extreme risks could reflect statistical overfitting, numerical error in an ill-conditioned fit, or both. To separate them, I refitted the 15 worst fpe realizations of each data set at $n = 30$ with two SVD-based solvers and one QR-based solver (Appendix, Check 4). For the 22 full-rank fits among these 30, the three risks agreed within 0.5%, even with condition numbers near 10^{14}. Rank-deficient fits make up 4 of the 300 fpe selections for pure noise and 9 for sine-squared. Among the 8 such fits in the two worst-15 lists, six agreed within 4.4%, and two sine-squared fits differed by 34% and by a factor of 15. The tail therefore reflects overfitting with an ill-conditioned basis, and numerical error changes the risk of a few rank-deficient fits.

Effect of sample size ($n = 100$). With $n = 100$, every method improves. For pure noise, vc selects the constant model in all 300 realizations, while fpe, gcv, and cv level off at 78–81%. For small p, these three criteria charge about $2\sigma^2/n$ for each extra parameter, and a spurious term beats that charge with a probability that does not shrink as n grows. The VC factor charges twice as much at the same p (2.47 against 1.22 for fpe at $p = 0.1$ with $n = 100$). For sine-squared, the median DoF rises to 9 for fpe, gcv, and cv, and the larger sample supports a closer approximation of the target. The four median risks lie between 0.0050 and 0.0052, and vc has the lowest 95th-percentile risk (0.0108). The textbook makes the same observation for its algebraic-polynomial experiment with 100 samples: most methods perform well, and vc keeps the lowest variability.

Conclusion

Across both data sets and both sample sizes, the VC penalization had the lowest 95th-percentile risk. Its lead is widest at $n = 30$. The methods differ more in the upper tail than at the median: for pure noise at $n = 30$ the medians of fpe and vc differ by a factor of 3.6 and their 95th percentiles by about 1200, and for sine-squared at $n = 100$ the four medians lie between 0.0050 and 0.0052. For this reason I rank the methods by their 95th-percentile risk, which the textbook calls the *guaranteed* prediction risk and uses to compare model selection methods.

Problem 2: Trading international mutual funds

Data and encoding

I use daily closing prices of the S&P 500 (`^GSPC`) and the Fidelity International Discovery fund (FIGRX) from Yahoo Finance, and the euro exchange rate (euros per US dollar) from the UBC Pacific Exchange Rate Service. Following the course preprocessing note, I keep the dates present in all three series: 243 trading days in 2004 and 245 in 2005. Alignment removes nine 2004 days on which the UBC file has no quote, most of them Canadian holidays, and seven US market holidays on which it does.

The inputs are today's percentage changes, x_1 for the S&P 500 and x_2 for the exchange rate,

each measured against the previous aligned day. The output y is tomorrow's percentage change of FIGRX. I join the price series across years to compute the changes at the year boundaries. The output for December 31, 2004 uses the first aligned day of 2005 (January 4). The input for January 6, 2004, the first aligned day of that year, needs the close of December 31, 2003, which the provided files do not contain; I retrieved the three values from the same sources (S&P 500 1111.92, FIGRX 23.92, 0.79386 euros per dollar). I obtain 243 training samples (January 6 to December 31, 2004) and 244 test samples (January 4 to December 29, 2005). The last day of 2005 has no next-day output.

FIGRX closed unchanged on seven 2004 days. I drop those days when I fit the quadratic classifier and when I compute accuracy. The trading simulation keeps them: a zero change produces zero gain under either position.

Classifiers

(a) Linear classifier. I fit $t(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2$ to the real-valued outputs by least squares and predict UP when $t(\mathbf{x}) > 0$. The 2004 data give

$$t(\mathbf{x}) = 0.0469 + 0.0844\, x_1 - 0.0073\, x_2.$$

The positive S&P 500 coefficient fits the view that foreign markets follow the US market with a one-day lag. The exchange-rate coefficient is close to zero. With a positive intercept, the rule predicts DOWN after the S&P 500 falls by more than about 0.56%, and it predicts UP on 79% of training days.

(b) Quadratic classifier. I convert the outputs to class labels $y = \text{sign}(y)$ and fit

$$g(\mathbf{x}, \mathbf{w}) = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2 + w_5 x_1 x_2 + w_0$$

to the ± 1 labels by least squares, then predict UP when $g > 0$. The assignment leaves the estimation method open; least squares keeps the two approaches comparable. The fitted coefficients are $w_0 = 0.1594$, $w_1 = 0.0787$, $w_2 = 0.1112$, $w_3 = -0.0606$, $w_4 = -0.1470$, and $w_5 = 0.1361$. The quadratic part is negative definite ($w_3 < 0$ and $w_3 w_4 - w_5^2/4 = 0.0043 > 0$) and the intercept is positive, so the boundary is an ellipse with the UP region inside. The model predicts UP on calm days and DOWN after large moves in either input.

Classifier	Training accuracy (2004)	Test accuracy (2005)
Linear	51.7%	57.1%
Quadratic	57.2%	55.8%
Always UP	52.5%	55.0%

The quadratic model gains 5.5 points of training accuracy over the linear one and loses 1.3 points on the test year, a pattern consistent with overfitting. The two models also differ in their fitting target (real-valued changes versus ± 1 labels), so this comparison mixes complexity with the choice of target. With about 240 days per year, one day moves accuracy by 0.4 points, and the standard error of an accuracy estimate is close to 3 points. Neither classifier beats the always-UP rule by more than that margin.

Decision boundaries

Figure 3 plots both boundaries over the training and test data in the required $(-2\%, +2\%)$ range. The linear boundary runs close to vertical at $x_1 \approx -0.56\%$, because the exchange rate carries almost no weight. The quadratic ellipse tilts toward the upper right and extends past the plotted range. Its lower edge near $x_2 = -0.7\%$ wraps around a group of 2004 DOWN days that has no counterpart in 2005. In all four panels the UP and DOWN days overlap, and no boundary of either shape separates them.

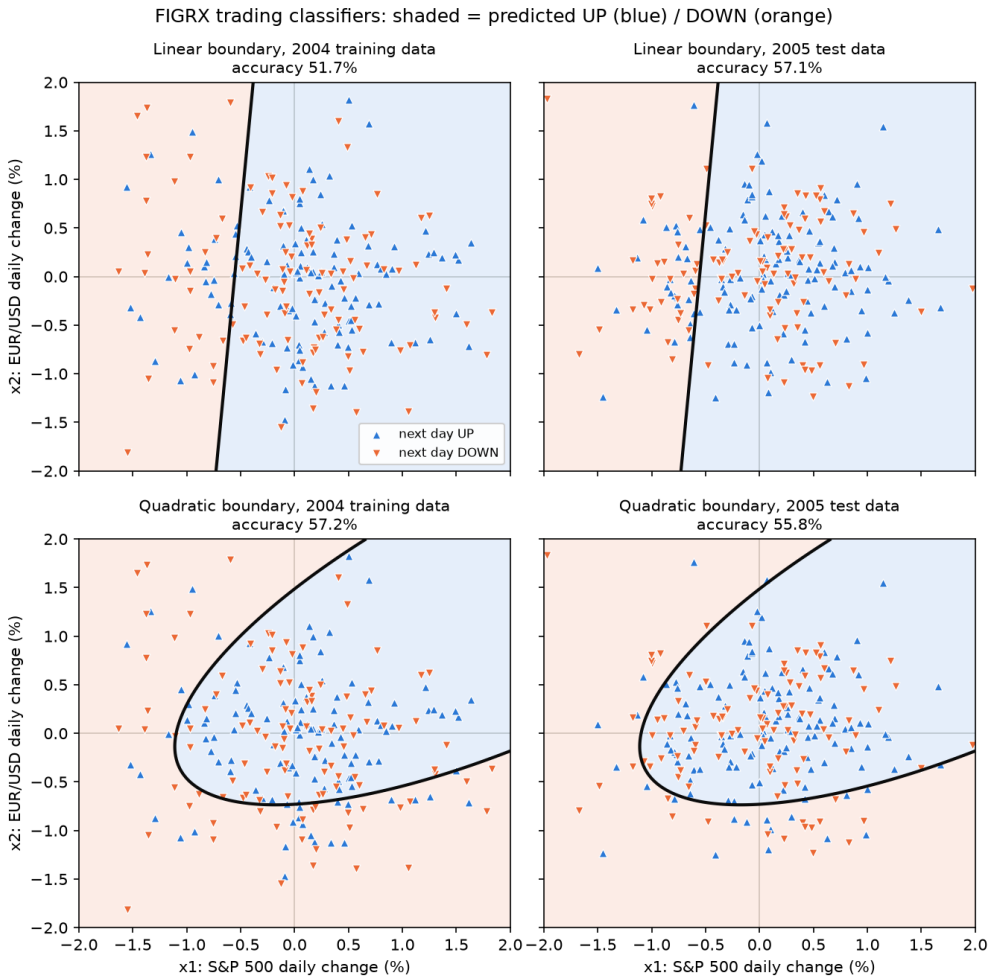


Figure 3: Decision boundaries of the linear (top) and quadratic (bottom) classifiers over the 2004 training data (left) and the 2005 test data (right). Shading shows the predicted class; markers show the actual next-day FIGRX change.

On 2005 days inside the ellipse, FIGRX rose the next day 56.6% of the time, against 47.7% outside. The outside group holds 44 days, so its standard error is about 7.5 points, and the gap falls within sampling error. On the 2004 data used to fit the ellipse, the same comparison gave 56.7% and 41.5%.

Trading results

Each strategy holds FIGRX on days its classifier predicts UP and cash otherwise. Starting from $1 million, I compound the daily returns over the 2005 test period. GAIN is the cumulative return at the end of the year, and EXPOSURE is the share of days invested in FIGRX.

Strategy	GAIN	EXPOSURE
Linear	21.81%	76.6%
Quadratic	15.19%	82.0%
Buy-and-Hold	14.92%	100.0%

Buy-and-Hold earns 14.92%, from the January 4 close (27.55) to the December 30 close (31.66). The linear strategy earns 6.9 points more while staying in cash on 57 days. On those days FIGRX moved -0.10% on average, against $+0.11\%$ on the days the strategy held the fund. The gain depends on the size of each day's move, so a rule with a small accuracy edge can earn more by skipping days with large declines. To judge luck, I drew 10,000 random strategies invested on the same 76.6% of days. Their median gain is 11.2%, and 3.1% of them reach 21.81% or more.

Discussion

(a) Can this strategy be used in practice? I would not trade my own money with it. The 2005 result rests on one test year, and I picked the better of two models after seeing both. Neither classifier beats the always-UP rule in accuracy by more than the sampling error, and the UP and DOWN days overlap across the input plane. A possible source of the linear rule's profit is the stale pricing of international funds, whose prices use foreign closes that have not yet reacted to the US market; my experiment does not test this mechanism. After the 2003 mutual-fund scandal, fund companies adopted fair-value pricing, and many restrict short-term trading or charge redemption fees, which would close such a gap. The assignment also assumes zero transaction costs. The linear strategy switches position 89 times in 2005. If each switch costs 0.1% of the account, its gain falls from 21.81% to 11.43%, below the 14.92% of Buy-and-Hold.

(b) VC-theoretical explanation. Problem 2 is a classification task with 0/1 loss, so the relevant result is the VC bound for classification. It adds a confidence term Φ to the training error rate,

$$R(\omega) \le R_{\text{emp}}(\omega) + \Phi, \qquad \Phi = \sqrt{\frac{h\big(\ln(2n/h)+1\big) - \ln(\eta/4)}{n}},$$

and holds with probability at least $1-\eta$. I set $\eta = \min(4/\sqrt{n}, 1)$, the textbook's practical rule, which raises the confidence level as the sample grows. The multiplicative VC penalization factor used in Problem 1 belongs to regression. Each classifier is linear in its parameters, so its VC-dimension equals the parameter count: $h = 3$ for the linear rule and $h = 6$ for the quadratic rule. A VC bound holds for every function in the model set, so it also covers my least-squares classifiers, which do not minimize the 0/1 training error.

Classifier	h	n	n/h	R_{emp}	Φ	Upper bound on R
Linear	3	236	78.7	0.483	0.298	0.781
Quadratic	6	236	39.3	0.428	0.385	0.813

The 0/1 training error uses the 236 days with a nonzero FIGRX change. With $n = 236$, the rule gives $\eta = 4/\sqrt{236} \approx 0.26$, so the bounds hold with probability at least $1 - \eta \approx 74\%$, below the 95% level common in statistics.

The textbook calls a sample small when n/h is below about 15; in that regime Φ stays large, and model selection has to match the complexity to the data. The assignment fixes two low-capacity model sets in advance and fits each once, without searching over model sizes. Both classifiers lie well above the small-sample threshold (n/h of 78.7 and 39.3), which gives intuitive support for this choice: further complexity control has less to gain here than in the small samples of Problem 1. This argument is a heuristic, and a large n/h leaves room for some benefit from complexity control.

Despite the large n/h, both upper bounds (0.781 and 0.813) exceed the 0.5 error of random guessing, so the theory certifies no useful classifier from these 236 days. The training error rates sit near 0.5 before adding Φ, and Φ needs a large n to become small, since its numerator also grows with $\ln n$. With $h = 3$ and the same rule for η, Φ falls to about 0.06 only near $n = 10{,}000$ trading days.

The bound also assumes independent samples from one fixed distribution. Daily returns can depend on earlier days, and the 2004 and 2005 markets need not share a distribution, so the bound gives a rough guide to the 2005 test error at best. My results show that these two classifiers, with these two inputs, did not deliver a robust predictive advantage in 2005; other models and other inputs remain untested.

Appendix: reproducing the numbers

Four scripts in the folder `HW2/practice` print the numbers in this report. `p1_mine.py` and `p2_mine.py` run the two experiments, print the result tables, and draw the figures. `p1_checks.py` prints the numerical checks below and the other Problem 1 values quoted in the text: the training-error example, the penalty factors, and the DoF chosen in the worst realizations. `p2_checks.py` prints the Problem 2 values the main script leaves out: the alignment counts, the shape test for the ellipse, the next-day UP rates inside and outside it, the mean FIGRX change on cash days, the gain after transaction costs, the sampling errors, and the values of Φ for larger samples. The two check scripts save their output in `p1_checks_output.txt` and `p2_checks_output.txt`.

Numerical checks for Problem 1

I ran all scripts with Python 3.11.9, NumPy 2.4.6, and SciPy 1.17.1. All checks use the data generator and random seeds of the main experiment. A matrix counts as rank deficient when NumPy's `matrix_rank` returns fewer than DoF; its threshold is the largest singular value times $\max(n, \text{DoF})$ times machine epsilon.

1. **Rank.** 200 samples of $x \sim U[0, 1]$ with $n = 30$ (seed 1); count of rank-deficient matrices for each candidate DoF.

2. **Leverage formula.** One sine-squared sample with $n = 30$ (seed 0); leave-one-out MSE from the formula and from 30 explicit refits, for DoF 9, 13, 15, 17, and 19.

3. **cv selections.** The 300 realizations of each setting (seed 0), with cv computed by my default rule and by explicit refits for every candidate DoF. I compare the selected DoF realization by realization, so "the same" means an identical choice in every one of the 300 realizations.

4. **Solvers.** The 15 realizations with the highest fpe risk for each data set at $n = 30$ (seed 0), refitted with the LAPACK drivers `gelsd` and `gelss` (SVD-based) and `gelsy` (QR with column pivoting) through SciPy's `lstsq` with its default cutoff; the output lists the risk under each driver.